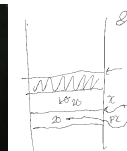


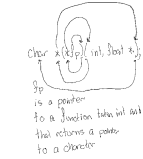
```
#include <iostream>
int main() {
    using namespace std;
    int w = 0;
    char s2[] = "Hello";
    char* p = s2;
    *p = '2';
    cout << "s2 = " << s2 << endl;
}
```

- What is happening ?



ex)


```
graph TD
    head((x)) --> node1((char))
    node1 --> node2((x))
    node2 --> node3((char))
    node3 --> node4((x))
    node4 --> node5((char))
    node5 --> node6((x))
    node6 --> node7((char))
    node7 --> node8((x))
    node8 --> node9((char))
    node9 --> node10((x))
    node10 --> node1
```



C++ pointers are strongly typed
This means if you have a pointer to
a int, that same pointer cannot point
to a char
you have to follow your pointer types
but we can go back to the C way of doing things

1. $\text{static_const} <T> (e)$
2. $\text{const_const} <T> (e)$
3. $\text{constexpr_const} <T> (e)$
4. $\text{dynamic_const} <T> (e)$

C: only one way
(1 + x) * (x)

→ Static cast

```
#include <iostream>

int main()
{
    int x = 10;
    char c = 'a';
    int* p = static_cast<int*>(&c);
    return 0;
}
```

pointer to a pointer of integer
 will error because we cannot convert pointer to char to pointer of integer

- purpose: change constness of expression
- when: done at compile time

```
#include <iostream>
int main() {
    using namespace std;
    const int x = 10;
    int* p = const_cast<int*>(&x);
    *p = 20;
    cout << "p = " << p << endl;
    cout << "x = " << &x << endl;
    cout << "x = " << x << endl;
    cout << "p = " << *p << endl;
    return 0;
}
```

It will allow us to
change what the const
value that we have stored
using pointers
It is assigning a pointer value to 20
mean it is not changing the value of x.

What checks?

- purpose: convert between compatible types in class hierarchies
- when: check done at runtime

STL
Standard template library

- containers
- iterators
- algorithms
- *glue (adapters)*

The diagram shows a rectangular container divided into four horizontal sections. The top section contains 'x', the second contains 'y', the third contains 'z', and the bottom contains 'w'. An arrow points from the left towards the container, and another arrow points upwards from below the container.

genre

- consider making a deal
- integers
- double
- characters
- strings
- students

7 some function

```
template <class T>
T sum(vector<T>& vec) {
    T ttl;
    for(T v : vec) ttl += v;
    return ttl;
}
```

This will not be fiction game
because it will work with hit and clock

$$\begin{aligned} k &= 0 \\ k &= n \\ k &= i \\ k &= j \\ k &= - \\ k &= n \end{aligned}$$

CON
1
1
1

container
iterator pointing at the right
most in a container. For not so backward

- Iterator is an object representing an index
- can move forward
- lookup the elements
- Can be compared with iterators
{ ex. to test end }

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> vec;
    for(int i=0; i<10; i++)
        vec.push_back(i);
    for(int i=0; i < vec.size(); i++) {
        cout << vec[i] << " ";
    }
    cout << endl;
    return 0;
}
```

the reverse

```
for(vector<int>::iterator k = vec.rbegin();
    k != vec.rend();
    k++) {
    cout << *k << " ";
}

return 0;
```

The *begin* is end and the *end* is the *beginning*

- Containers provide
 - vector :: linear
 - vector :: random access
 - map :: linear direction
 - multiset :: to obtain nodes at either end
 - forward and reverse
 - end points to the last element

you can construct nodes keeps here

- You can use it almost everywhere
 - For local variable declaration
 - For return types of function
- It makes the code completely generic
- But...
 - Not for argument types

```
template <class Container>
void print(Container& c) {
    for(auto& k = c.begin(); k != c.end(); k++) {
        cout << *k << " ";
    }
    cout << endl;
}

int main() {
    vector<int> v;
    for(int i=0; i<10; i++)
        v.push_back(i);
    print(v);
    return 0;
}
```

Example

```

template<class Container>
void print(Container& c) {
    for(auto& x : c) cout<< x<< " ";
}

int main() {
    Container c1;
    Container c2;
    print(c1);
    print(c2);
    return 0;
}

```

same code

different container

- As long as you have a good sense of the type
- So be careful as you learn!

```
int main() {
    using namespace std;

    vector<int> v = {1,10,5,12,34,17,51,63,99,1,6,100};
    auto it = next(it, 5);
    cout << *it << endl;
    it++;
    cout << *it << endl;
    return 0;
}
```

[illegible]

- Adaptors
 - create primitive alphas
 - sort
 - combinatorial algorithm
 - has counters A and B
 - inserts at the end of B everything that is in A

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    // declare vector
    for(vector<int>::iterator k = vec.begin();
        k != vec.end(); k++) {
        cout << *k << " ";
    }
    cout << endl << "Reverse!" << endl;
    return 0;
}
```

```
int main() {
    using namespace std;
    vector<int> a = {1,2,3,4,5};
    vector<int> b = {10,11,12,13};

    insertInto(b,a.begin(),a.end());

    for(int v : b)
        cout << v << " ";
    cout << endl;
    return 0;
}
```

```
template <class Container, class InputIterator>
void insertInto(Container &b, InputIterator from, InputIterator to)
{
    while (from != to) {
        b.push_back(*from);
        ++from;
    }
}
```